MQTT Payload 규격

MQTT Payload 규격

최종 업데이트: 2026-02-25

변경 이력

날짜	내용
2026-02-25	controller 필드에 damper_ctrl 추가. 댐퍼 제어 명령값(Damper_CTRL) 전달하여 컨트롤 패널에서 명령값 표시
2026-02-25	pp_spark 범위 확대 0-99 → 0-9999 (파워팩 rev2.1 대응). 알람 임계값 ≥99 → ≥9999
2026-02-21	controller 필드에 fan_freq, fan_target_pct 추가. AUTO(PID) 모드에서 인버터 실제 출력 주파수와 PID 목표값 모니터링용
2026-02-21	oil_level 타입 변경: float(0-100%) → int(0=정상, 1=만수). 디지털 센서 실제 동작에 맞게 수정
2026-02-17	controller 필드에 fan_running 추가, status_flags bit 5 판정 로직 변경 (통신+고장 복합 판정)
2026-02-15	status 토픽에 wifi 객체 추가 (ssid, rssi, ip, mac, channel)
2026-02-13	댐퍼/시로코팬 자동 제어 명령 추가 (모드 전환, 목표 풍량/풍속 설정), fan_mode/damper_mode 센서 필드 추가
2026-02-12	config 토픽 페이로드 정의, config/ack 응답 추가
2026-02-12	controller 필드에 flo-OAC 유입온도(inlet_temp), 풍속(velocity), 덕트차압(duct_dp) 추가
2026-02-11	ID 할당 방식 설명 추가, 일괄 제어 범위 지정 방식 보완
2026-02-11	QoS/Retain 정보 추가, equipment_id 필터링 명확화
2026-02-11	통합 센서 메시지 구조 도입, ID 문자열 통일, 토픽 간소화, status_flags 재설계 (Gateway 7비트/Controller 6비트), blade_angle 제거 (damper로 통합), pp_temp 타입 int 명확화, voc/nox null 명시, 알람 토픽 제거, error_code·fan_rpm 제거
2026-02-03	초안 작성

계층 구조

```
Site (매장)
└── Floor (층)
    └── Gateway (게이트웨이, 층당 1대)
        └── Equipment (집진기, 층당 N대)
            └── Controller (파워팩, 집진기당 1~3대)
```

- 1개 Site에 여러 Floor
- 1개 Floor에 1개 Gateway
- 1개 Gateway에 최대 5대 Equipment (집진기)
- 1개 Equipment에 최대 4대 Controller (파워팩)

ID 형식

모든 ID는 문자열(string) 타입입니다.

대상	형식	예시	설명
site_id	string	`"site-001"`	매장 식별자
floor_id	string	`"1F"`, `"B1"`	층 식별자
gateway_id	string	`"gw-001"`	게이트웨이 식별자
equipment_id	string	`"esp-001"`	집진기(장비) 식별자
controller_id	string	`"ctrl-001"`	컨트롤러(파워팩) 식별자

ID 할당 방식

모든 ID는 원청 내부 고유번호를 사용하며, 설치 시 각 장비에 설정합니다.

- Controller: 컨트롤러 펌웨어에 `controller_id` + `equipment_id` 설정. 페어링 시 게이트웨이에 전달.
- Gateway: 게이트웨이 펌웨어에 `gateway_id`, `site_id`, `floor_id` 설정.
- 게이트웨이는 컨트롤러로부터 수신한 `equipment_id`로 자동 그룹핑하여 MQTT에 발행.

토픽 구조

게이트웨이 단위로 토픽을 통합합니다. 센서/상태/제어 모두 게이트웨이 토픽 하위에 위치합니다.
알람은 별도 토픽 없이 센서 데이터의 `pp_alarm` 필드로 전달됩니다.

```
metabeans/{site_id}/{floor_id}/gateway/{gw_id}/
├── sensor          # 통합 센서 데이터 발행 (10초 주기)
├── status          # 게이트웨이 상태 발행 (10초 주기)
├── control         # 제어 명령 수신 (구독)
```

```
├── control/ack      # 제어 명령 응답 발행
├── config           # 설정 변경 수신 (구독)
└── config/ack       # 설정 변경 응답 발행
```

토픽 예시

- 센서 데이터: `metabeans/site-001/1F/gateway/gw-001/sensor`
- 제어 명령: `metabeans/site-001/1F/gateway/gw-001/control`
- 제어 응답: `metabeans/site-001/1F/gateway/gw-001/control/ack`

구독 패턴 예시

- 특정 매장 전체: `metabeans/site-001/+/gateway/+/#`
- 특정 층: `metabeans/site-001/1F/gateway/+/sensor`

발행 (Publish) 토픽

1. sensor — 통합 센서 데이터

토픽: `metabeans/{site_id}/{floor_id}/gateway/{gw_id}/sensor`
주기: 10초
방향: Gateway → Cloud

게이트웨이 IAQ 데이터와 모든 하위 equipment/controller 데이터를 하나의 메시지로 통합하여 발행합니다.
각 controller 데이터에는 개별 timestamp가 포함되어 게이트웨이가 마지막으로 수신한 시점을 클라우드에서 확인할 수 있습니다.

```json
{
  "gateway_id": "gw-001",
  "timestamp": 1234567890,
  "iaq": {
    "pm1_0": 12.5,
    "pm2_5": 25.0,
    "pm4_0": 30.0,
    "pm10": 35.0,
    "temperature": 24.5,
    "humidity": 65.0,
    "voc_index": 100,
    "nox_index": 50,
    "co2": 450,
    "o3": 25,
    "co": 1.2,
    "hcho": 30
  },
  "equipments": [
    {
      "equipment_id": "esp-001",
```

```
    "controllers": [
      {
        "controller_id": "ctrl-001",
        "timestamp": 1234567885,
        "pm2_5": 25.0,
        "pm10": 35.0,
        "diff_pressure": 12.0,
        "oil_level": 0,
        "pp_temp": 45,
        "pp_spark": 0,
        "pp_power": 1,
        "pp_alarm": 0,
        "fan_speed": 2,
        "fan_mode": 0,
        "fan_running": 1,
        "fan_freq": 30.00,
        "fan_target_pct": 50.0,
        "damper_mode": 0,
        "flow": 850.0,
        "damper_ctrl": 80.0,
        "damper": 75.0,
        "inlet_temp": 22.5,
        "velocity": 8.3,
        "duct_dp": 245.0,
        "status_flags": 63
      },
      {
        "controller_id": "ctrl-002",
        "timestamp": 1234567888,
        "pm2_5": 18.0,
        "pm10": 28.0,
        "diff_pressure": 10.5,
        "oil_level": 0,
        "pp_temp": 42,
        "pp_spark": 0,
        "pp_power": 1,
        "pp_alarm": 0,
        "fan_speed": 2,
        "fan_mode": 0,
        "fan_running": 1,
        "fan_freq": 30.00,
        "fan_target_pct": 0.0,
        "damper_mode": 0,
        "flow": 820.0,
        "damper_ctrl": 80.0,
        "damper": 75.0,
        "inlet_temp": 21.8,
        "velocity": 7.9,
        "duct_dp": 230.0,
        "status_flags": 63
      }
    ]
  },
```

```
    {
      "equipment_id": "esp-002",
      "controllers": [
        {
          "controller_id": "ctrl-001",
          "timestamp": 1234567882,
          "pm2_5": 15.0,
          "pm10": 22.0,
          "diff_pressure": 8.0,
          "oil_level": 1,
          "pp_temp": 38,
          "pp_spark": 0,
          "pp_power": 1,
          "pp_alarm": 0,
          "fan_speed": 1,
          "fan_mode": 0,
          "fan_running": 1,
          "fan_freq": 15.00,
          "fan_target_pct": 0.0,
          "damper_mode": 0,
          "flow": 600.0,
          "damper_ctrl": 65.0,
          "damper": 60.0,
          "inlet_temp": 23.1,
          "velocity": 5.2,
          "duct_dp": 180.0,
          "status_flags": 63
        }
      ]
    }
  ]
}
```


최상위 필드

필드	타입	설명
gateway_id	string	게이트웨이 ID
timestamp	int	메시지 발행 시간 (Unix epoch, 초)
iaq	object	게이트웨이 IAQ 센서 데이터
equipments	array	하위 집진기 목록

iaq 필드

필드	타입	단위	설명
pm1_0	float	μg/m³	PM1.0 농도
pm2_5	float	μg/m³	PM2.5 농도

필드	타입	단위	설명
pm4_0	float	µg/m³	PM4.0 농도
pm10	float	µg/m³	PM10 농도
temperature	float	°C	온도
humidity	float	%	습도
voc_index	int | null	-	VOC 지수 (1-500), 센서 워밍업 중 null
nox_index	int | null	-	NOx 지수 (1-500), 센서 워밍업 중 null
co2	int	ppm	CO2 농도
o3	int	ppb	오존 농도
co	float	ppm	일산화탄소 농도
hcho	int	ppb	포름알데히드 농도

equipments[].controllers[] 필드

필드	타입	단위	설명
controller_id	string	-	컨트롤러 ID
timestamp	int	epoch, 초	게이트웨이가 해당 컨트롤러 데이터를 마지막으로 수신한 시간
pm2_5	float	µg/m³	배출부 PM2.5 농도
pm10	float	µg/m³	배출부 PM10 농도
diff_pressure	float	Pa	ESP 집진부 차압
oil_level	int	-	오일 만수 감지 (0=정상, 1=만수)
pp_temp	int	°C	파워팩 온도 (정수, x10 스케일 아님)
pp_spark	int	-	스파크 수치 (0-9999, rev2.1 이전 0-99)
pp_power	int	-	전원 상태 (0=OFF, 1=ON)
pp_alarm	int	-	파워팩 알람 (0=정상, 1=알람)
fan_speed	int	-	팬 속도 단계 (0=OFF, 1=LOW, 2=MID, 3=HIGH), 수동 모드에서만 유의미
fan_mode	int	-	팬 제어 모드 (0=수동, 1=자동)
fan_running	int	-	인버터 실제 운전 상태 (0=정지, 1=운전 중). M100 RUN_STATUS 레지스터 기반
fan_freq	float	Hz	M100 인버터 실제 출력 주파수 (0~50.00Hz). 자동 모드에서 PID 출력 확인용
fan_target_pct	float	%	PID 목표값 (0.0~100.0%). fan_mode=1(자동)일 때만 유의미, 수동 모드에서는 0
damper_mode	int	-	댐퍼 제어 모드 (0=수동, 1=자동)
flow	float	CMH	풍량 (flo-OAC Q_act, 현재유량)

필드	타입	단위	설명
damper_ctrl	float	%	댐퍼 제어 명령값 (flo-OAC Damper_CTRL, 0-100)
damper	float	%	댐퍼 개도율 (flo-OAC Damper_FB 피드백, 0-100)
inlet_temp	float	°C	유입 온도 (flo-OAC T_act, -20~50)
velocity	float	m/s	현재 풍속 (flo-OAC V_act, 0~20.0)
duct_dp	float	Pa	덕트 차압 (flo-OAC DP_Pv, -49~980)
status_flags	int	-	컨트롤러 상태 플래그 (비트마스크, 아래 표 참조)

status_flags 비트 정의 (Controller)

비트	의미
0	파워팩 RS-485 통신 정상
1	SPS30 (PM2.5) 센서 정상
2	SDP810 (차압) 센서 정상
3	수위 센서 정상
4	flo-OAC 댐퍼 컨트롤러 정상
5	LS M100 인버터 정상 (RS-485 통신 정상 AND Fault Trip 없음)

페이로드 크기 예상

구성	예상 크기
Gateway IAQ만	~300 bytes
Equipment 1대 x Controller 2대	~600 bytes
Equipment 3대 x Controller 4대	~3.5 KB
최대 (Equipment 5대 x Controller 4대)	~6 KB

MQTT 메시지 크기 제한(128KB) 대비 충분합니다.

---

2. status — 게이트웨이 상태

토픽: `metabeans/{site_id}/{floor_id}/gateway/{gw_id}/status`
주기: 10초 (sensor와 동시)
방향: Gateway → Cloud

```
{
  "gateway_id": "gw-001",
```

```
  "status_flags": 63,
  "controller_count": 3,
  "wifi": {
    "ssid": "factory-wifi",
    "rssi": -45,
    "ip": "192.168.1.100",
    "mac": "AA:BB:CC:DD:EE:FF",
    "channel": 6
  },
  "timestamp": 1234567890
}
```

필드	타입	설명
gateway_id	string	게이트웨이 ID
status_flags	int	상태 플래그 (비트마스크, 아래 표 참조)
controller_count	int	현재 연결된 컨트롤러 수
wifi	object	Wi-Fi 연결 정보
timestamp	int	발행 시간 (Unix epoch, 초)

wifi 필드

필드	타입	설명
ssid	string	연결된 AP의 SSID
rssi	int	신호 강도 (dBm, 일반적으로 -30 ~ -90)
ip	string	게이트웨이 IP 주소
mac	string	게이트웨이 MAC 주소 (XX:XX:XX:XX:XX:XX 형식)
channel	int	Wi-Fi 채널 (1-14, 미연결 시 0)

Wi-Fi 미연결 시 `ssid`, `ip` 필드가 누락될 수 있으며, `rssi`는 0, `channel`은 0으로 발행됩니다.

status_flags 비트 정의 (Gateway)

비트	의미
0	SEN55 정상 (PM, 온습도, VOC, NOx)
1	SCD40 정상 (CO2)
2	O3 센서 정상 (SEN0321)
3	CO 센서 정상 (SEN0466)
4	HCHO 센서 정상 (SFA30)
5	1개 이상 컨트롤러 연결됨
6	페어링 모드

3. control/ack — 제어 명령 응답

토픽: `metabeans/{site_id}/{floor_id}/gateway/{gw_id}/control/ack`
방향: Gateway → Cloud

```
{
  "cmd_id": "550e8400-e29b-41d4-a716-446655440000",
  "result": "success",
  "reason": ""
}
```

필드	타입	설명
cmd_id	string	명령 ID (요청과 매칭)
result	string	`"success"` 또는 `"fail"`
reason	string	실패 시 사유 (성공 시 빈 문자열)

---

4. config/ack — 설정 변경 응답

토픽: `metabeans/{site_id}/{floor_id}/gateway/{gw_id}/config/ack`
방향: Gateway → Cloud

```
{
  "cmd_id": "550e8400-e29b-41d4-a716-446655440002",
  "result": "success",
  "reason": "",
  "needs_reboot": true
}
```

필드	타입	설명
cmd_id	string	명령 ID (요청과 매칭)
result	string	`"success"` 또는 `"fail"`
reason	string	실패 시 사유 (성공 시 빈 문자열)
needs_reboot	bool	`true` 이면 ACK 발행 후 자동 재부팅 예정

구독 (Subscribe) 토픽

1. control — 제어 명령

토픽: `metabeans/{site_id}/{floor_id}/gateway/{gw_id}/control`
방향: Cloud → Gateway

개별 컨트롤러 제어

```
{
  "cmd_id": "550e8400-e29b-41d4-a716-446655440000",
  "equipment_id": "esp-001",
  "controller_id": "ctrl-001",
  "target": 0,
  "action": 1,
  "value": 0
}
```

일괄 제어

`equipment_id` 와 `controller_id` 조합으로 제어 범위를 지정합니다.

equipment_id	controller_id	범위
`"all"`	`"all"`	게이트웨이 하위 전체 컨트롤러
`"esp-001"`	`"all"`	해당 집진기 하위 컨트롤러만
`"esp-001"`	`"ctrl-001"`	특정 컨트롤러 지정

```
{
  "cmd_id": "550e8400-e29b-41d4-a716-446655440001",
  "equipment_id": "all",
  "controller_id": "all",
  "target": 0,
  "action": 1,
  "value": 0
}
```

필드	타입	설명
cmd_id	string	명령 ID (ACK 매칭용, 필수, UUID 권장)
equipment_id	string	대상 집진기 ID (`"all"` = 전체 집진기)
controller_id	string	대상 컨트롤러 ID (`"all"` = 전체 컨트롤러)
target	int	제어 대상 (아래 표 참조)
action	int	액션 코드 (아래 표 참조)
value	number	설정값 (타겟/액션에 따라 의미 다름, int 또는 float)

제어 대상 (target) 및 액션 (action) 정의

target=0: 파워팩 (Powerpack)

action	value	설명
0	-	파워팩 OFF
1	-	파워팩 ON
2	-	파워팩 리셋

target=1: 댐퍼 (Damper / flo-OAC)

action	value	설명
1	0-100 (int)	댐퍼 개도율 설정 (%, 수동 모드)
2	0 또는 1 (int)	제어 모드 전환 (0=수동, 1=자동)
3	float (CMH)	목표 풍량 설정 (자동 모드, 예: 850.0)

- action=1: flo-OAC를 Manual 모드로 전환 후 개도율(%)을 직접 설정합니다. flo-OAC 하드웨어는 Float 0~100% 연속 제어를 지원하며, MQTT에서는 정수(0-100)로 전달하고 컨트롤러에서 float으로 변환합니다.
- action=2: 댐퍼 제어 모드를 전환합니다. 자동 모드(1)로 전환하면 flo-OAC의 Internal SV 모드가 활성화되어, 설정된 목표 풍량을 기준으로 flo-OAC가 자체 PID로 댐퍼 개도를 자동 조절합니다.
- action=3: 자동 모드의 목표 풍량(CMH)을 설정합니다. 자동 모드가 아닌 경우 자동 모드로 자동 전환됩니다.

target=2: 시로코팬 (Fan / LS M100 인버터)

action	value	설명
0	-	팬 OFF (수동)
1	-	팬 LOW (수동, 15Hz)
2	-	팬 MID (수동, 30Hz)
3	-	팬 HIGH (수동, 50Hz)
4	0 또는 1 (int)	제어 모드 전환 (0=수동, 1=자동)
5	float (m/s)	목표 풍속 설정 (자동 모드, 예: 3.5)

- action=0~3: 기존 수동 속도 단계 제어입니다. 자동 모드 중 이 명령을 수신하면 자동으로 수동 모드로 전환됩니다.
- action=4: 팬 제어 모드를 전환합니다. 자동 모드(1)로 전환하면 M100 인버터의 내장 PID가 활성화되어, 설정된 목표 풍속을 기준으로 인버터가 자동으로 주파수(팬 속도)를 가/감속합니다. 피드백은 flo-OAC의 실측 풍속(V_act)을 사용합니다.
- action=5: 자동 모드의 목표 풍속(m/s)을 설정합니다. 자동 모드가 아닌 경우 자동 모드로 자동 전환됩니다.

자동 제어 동작 참고

- 댐퍼 자동 제어: flo-OAC 하드웨어가 자체 PID로 댐퍼 개도를 조절합니다. 컨트롤러 펌웨어는 목표 풍량 전달만 수행합니다.
- 팬 자동 제어: M100 인버터 내장 PID를 활용합니다. 컨트롤러 펌웨어가 flo-OAC에서 읽은 실측 풍속을 인버터 PID 피드백 레지스터에 주기적으로 전달하고, 인버터가 목표 풍속과의 오차를 기반으로 주파수를 자동 조절합니다.
- 자동→수동 전환: 모드 전환 명령(action=2 또는 4, value=0) 또는 수동 명령(댐퍼 action=1, 팬 action=0~3)을 보내면 자동 모드가 해제됩니다.
- 안전 오버라이드: 비상정지(ESTOP), 스파크 감지, 과온도 알람 발생 시 컨트롤러가 자동으로 팬/댐퍼 자동 모드를 해제(수동 전환)합니다. 이후 센서 데이터의 `fan_mode`, `damper_mode` 필드가 0으로 변경됩니다.
- PID 튜닝: 팬의 PID 게인(P/I/D)은 M100 인버터 파라미터(AP.22~AP.24)로 설정하며, 현장 환경에 따라 시운전 시 조정합니다.

---

2. config — 설정 변경

토픽: `metabeans/{site_id}/{floor_id}/gateway/{gw_id}/config`
방향: Cloud → Gateway

게이트웨이의 런타임 설정을 원격으로 변경합니다. 부분 업데이트(partial update)를 지원하며, 포함된 필드만 변경됩니다.

```
{
  "cmd_id": "550e8400-e29b-41d4-a716-446655440002",
  "site_id": "site-001",
  "floor_id": "1F",
  "gateway_id": "gw-002",
  "sensor_interval_ms": 5000,
  "mqtt_interval_ms": 10000,
  "mqtt_broker_uri": "mqtts://new-broker.example.com:8883",
  "wifi_ssid": "NewNetwork",
  "wifi_password": "newpass123",
  "reboot": true
}
```

필드	타입	필수	설명
cmd_id	string	필수	명령 ID (config/ack 매칭용, UUID 권장)
site_id	string	선택	매장 ID 변경 (NVS 저장, 재부팅 필요)
floor_id	string	선택	층 ID 변경 (NVS 저장, 재부팅 필요)
gateway_id	string	선택	게이트웨이 ID 변경 (NVS 저장, 재부팅 필요)
sensor_interval_ms	int	선택	센서 폴링 주기 (ms, 1000~60000, 기본 5000)
mqtt_interval_ms	int	선택	MQTT 발행 주기 (ms, 5000~60000, 기본 10000)
mqtt_broker_uri	string	선택	MQTT 브로커 URI (NVS 저장, 재부팅 필요)
wifi_ssid	string	선택	Wi-Fi SSID (NVS 저장, 재부팅 필요)
wifi_password	string	선택	Wi-Fi 비밀번호 (NVS 저장, 재부팅 필요)

필드	타입	필수	설명
reboot	bool	선택	`true` 시 즉시 재부팅 수행

필드 분류

즉시 적용 (재부팅 불필요):

- `sensor_interval_ms` — 다음 센서 폴링 주기부터 적용
- `mqtt_interval_ms` — 다음 MQTT 발행 주기부터 적용

NVS 저장 + 재부팅 필요:

- `site_id`, `floor_id`, `gateway_id` — MQTT 토픽 경로 변경
- `mqtt_broker_uri` — MQTT 재연결 필요
- `wifi_ssid`, `wifi_password` — Wi-Fi 재연결 필요

자동 재부팅 동작

재부팅이 필요한 필드(`site_id`, `floor_id`, `gateway_id`, `mqtt_broker_uri`, `wifi_ssid`, `wifi_password`)가 변경되면:

1. NVS에 새 값 저장
2. config/ack 응답 발행 (`needs_reboot: true`)
3. 1초 대기 후 자동 재부팅 수행

`reboot: true`가 명시적으로 포함된 경우에도 동일하게 ACK 후 1초 대기 재부팅합니다.

유효성 검증

필드	검증 규칙	실패 시
sensor_interval_ms	1000 ≤ value ≤ 60000	`"fail"` + 사유
mqtt_interval_ms	5000 ≤ value ≤ 60000	`"fail"` + 사유
문자열 필드	빈 문자열 불가	`"fail"` + 사유
cmd_id	누락 시	메시지 무시 (ACK 불가)

MQTT 통신 설정

QoS (Quality of Service)

AWS IoT Core는 QoS 0과 QoS 1만 지원합니다 (QoS 2 미지원).

토픽	QoS	설명
sensor	1	센서 데이터 누락 방지
status	1	상태 정보 누락 방지
control	1	제어 명령 전달 보장
control/ack	1	응답 전달 보장
config	1	설정 변경 전달 보장
config/ack	1	설정 응답 전달 보장

Retain

모든 발행 토픽의 Retain 플래그는 0 (비활성) 입니다. 상태 정보는 주기적(10초)으로 발행되므로 Retain이 불필요합니다.

참고 사항

타임스탬프

- 모든 timestamp는 Unix epoch (초 단위) 입니다.
- 서버/클라우드는 UTC로 저장하고, 클라이언트에서 로컬 시간으로 변환합니다.

센서 값 스케일

MQTT 페이로드의 모든 값은 변환 없이 그대로 사용할 수 있습니다. 펌웨어가 내부 스케일을 실제 값으로 변환한 뒤 전송합니다.

iaq 필드

필드	MQTT 타입	예시	비고
pm1_0, pm2_5, pm4_0, pm10	float	`25.3`	µg/m³, 소수점 1자리
temperature	float	`24.5`	°C, 소수점 1자리
humidity	float	`65.0`	%, 소수점 1자리
voc_index	int | null	`100`	1-500 정수, 워밍업 중 null
nox_index	int | null	`50`	1-500 정수, 워밍업 중 null
co2	int	`450`	ppm 정수
o3	int	`25`	ppb 정수
co	float	`1.2`	ppm, 소수점 1자리
hcho	int	`30`	ppb 정수

controllers[] 필드

필드	MQTT 타입	예시	비고
pm2_5, pm10	float	`25.0`	μg/m³, 소수점 1자리
diff_pressure	float	`12.0`	Pa, 소수점 1자리
oil_level	int	`1`	0=정상, 1=만수
pp_temp	int	`45`	°C 정수 (소수점 없음)
pp_spark	int	`0`	0-9999 정수 (rev2.1 이전 0-99)
pp_power	int	`1`	0=OFF, 1=ON
pp_alarm	int	`0`	0=정상, 1=알람
fan_speed	int	`2`	0=OFF, 1=LOW, 2=MID, 3=HIGH (수동 모드)
fan_mode	int	`0`	0=수동, 1=자동
fan_running	int	`1`	0=정지, 1=운전 중 (M100 실제 상태)
fan_freq	float	`30.00`	Hz, 소수점 2자리 (M100 출력 주파수)
fan_target_pct	float	`50.0`	%, 소수점 1자리 (PID 목표, 자동 모드)
damper_mode	int	`0`	0=수동, 1=자동
flow	float	`850.0`	CMH, 소수점 1자리
damper_ctrl	float	`80.0`	%, 소수점 1자리 (제어 명령값)
damper	float	`75.0`	%, 소수점 1자리 (피드백)
inlet_temp	float	`22.5`	°C, 소수점 1자리 (flo-OAC)
velocity	float	`8.3`	m/s, 소수점 1자리 (flo-OAC)
duct_dp	float	`245.0`	Pa, 소수점 1자리 (flo-OAC)
status_flags	int	`63`	비트마스크